INTERVIEW PACK · PORTFOLIO PROJECT 01

Ledgerline Compliance Copilot

A banking risk & compliance AI assistant (RAG) with citations, confidence scoring, human-in-the-loop review and a tamper-evident audit trail.

Live demo: https://abv2xygrggmaucidx5hjbg.streamlit.app

Code: https://github.com/nipunrg1990rocks-dev/banking-ai-portfolio

Author: Nipun Gupta · October 2026

Independent project developed with AI-assisted development tools. All policies and data are synthetic and describe a fictional bank. Not affiliated with any employer or real financial institution.

> How to describe it: "I developed this independent project using AI-assisted development tools. I defined the requirements and controls from my banking experience, directed the AI tools that wrote the code, reviewed the outputs, and tested and validated the result." Do not attribute it to any employer. Do not claim you hand-wrote the code.

---

1. The 60-second executive summary

"Banks run on policies. Analysts lose time searching KYC, AML and sanctions manuals, and a wrong answer can mean a regulatory breach. I built a compliance copilot that answers employees' policy questions **only from approved policy text**. Every answer cites the exact policy section and shows a confidence level. Anything that touches a regulated decision, such as releasing a sanctions hit, a tipping-off risk, or a request for a policy exception, is **routed to a human reviewer** with maker-checker control. Personal data is redacted before anything is processed. Prompt-injection attempts are blocked. Every step is written to a **hash-chained, tamper-evident audit trail**. It runs free in an offline mode that quotes policy text verbatim, so it can't hallucinate. A live LLM mode can be switched on when budget is approved. I measured it on a 37-question test set: the correct policy section was in the top three results every time, and every question was routed correctly."

2. Business problem and banking use case

- **Users:** first-line operations, KYC and AML analysts, relationship managers, and second-line compliance.
- **Pain:** policies are long, versioned and cross-referenced. Answers given from memory are inconsistent. Supervisors spend time answering repetitive questions. Regulators expect decisions to be traceable to policy.
- **Use case:** a self-service policy assistant that gives a grounded, cited answer in seconds and refuses or escalates when it shouldn't answer.
- **Value levers:** time saved per query, consistency, fewer policy breaches, and audit evidence. It also follows AI governance expectations: human oversight, explainability, logging and data minimisation.
- **Why the guardrails matter in banking:** an AI that tells an employee "yes, release the payment" or "you can tell the customer about the SAR" creates a sanctions breach or a tipping-off offence. The copilot is designed so that cannot happen.

3. What each technology does (plain English, then technical)

Technology	Plain English	Technical detail in this project
RAG (Retrieval-Augmented Generation)	Look up the relevant pages first, then answer only from them	`retriever.py` finds the top 5 policy sections. `answering.py` builds the answer only from those sections.

Technology	Plain English	Technical detail in this project
Chunking	Cutting documents into citable pieces	One chunk per numbered `##` section. Long sections are split on paragraphs with one paragraph of overlap (`documents.py`).
BM25	Classic keyword search scoring, like a search engine	`rank_bm25.BM25Okapi` over stemmed tokens. Good for exact terms such as "CTR" and "85 percent".
Embeddings / vector search	Turning text into numbers so similar meanings are close together	TF-IDF (word and two-word phrases) reduced to dense vectors with truncated SVD (Latent Semantic Analysis), compared with cosine similarity (`DenseIndex`).
Hybrid search + RRF	Combine two rankings fairly	Reciprocal Rank Fusion: score = Σ 1/(60 + rank) across the BM25 and dense rankings.
Query expansion	Teach the search engine banking jargon	Acronyms such as PEP, SAR and CET1, and everyday words such as "tell" → "disclose", are expanded before searching.
Stemming	Treat "filed", "filing" and "file" as the same word	Snowball stemmer (NLTK) in `portfolio_core/text.py`.
LLM (Claude)	The writing engine, used only in live mode	Called through the official Anthropic Python SDK with a grounded system prompt. Sources are numbered, and the model must cite [n] or reply `INSUFFICIENT_EVIDENCE`.
Citation validation	Check the AI didn't invent a source	A regex extracts [n] markers. Any missing or out-of-range citation triggers a fallback to the offline answer.
Confidence score	How sure the system is	60% term coverage + 40% semantic similarity, giving High, Medium or Low. Low means no answer is given and the question is referred to Compliance.
Guardrails	Safety checks before anything happens	PII regex redaction, prompt-injection patterns, and a length limit (`portfolio_core/guardrails.py`).
Human-in-the-loop	A person must sign off risky answers	SQLite review queue. Approve, amend or reject, with a mandatory rationale. The person who asked cannot review their own question.
Audit trail	Evidence of who did what, when	Every event stores SHA-256(previous hash + event). Editing or deleting any row breaks the chain, and `verify_chain()` reports where.
Streamlit	Python framework for web apps	Six tabs: Ask, Review queue, Audit trail, Policy library, Evaluation, How it works.
pytest + Playwright	Automated testing	48 unit and integration tests, plus a 10-step browser test that clicks through the real app.
GitHub Actions	Runs the tests on every change	`.github/workflows/ci.yml`: lint, tests, and the evaluation report.

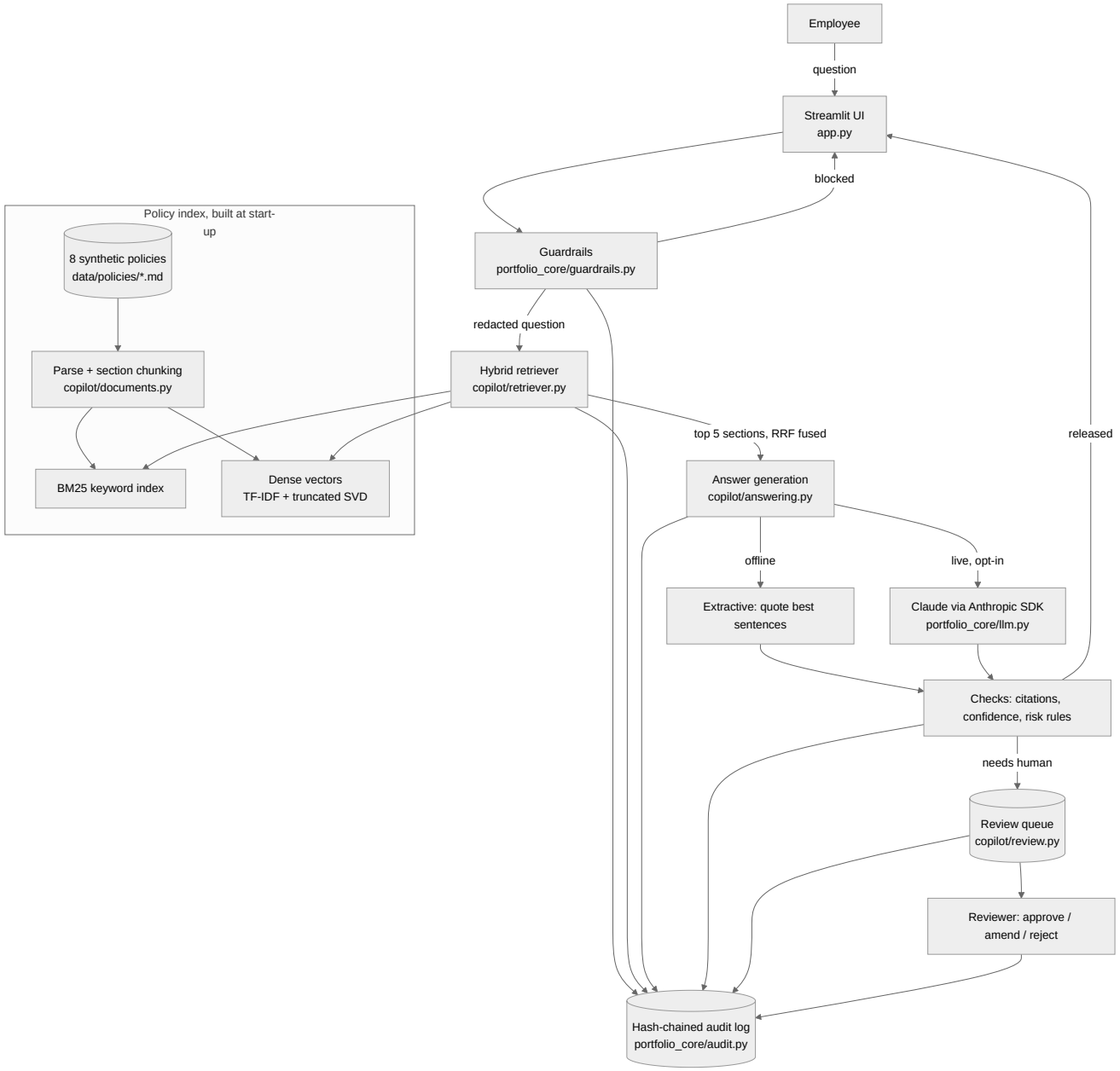
4. Architecture and diagrams

1. Context

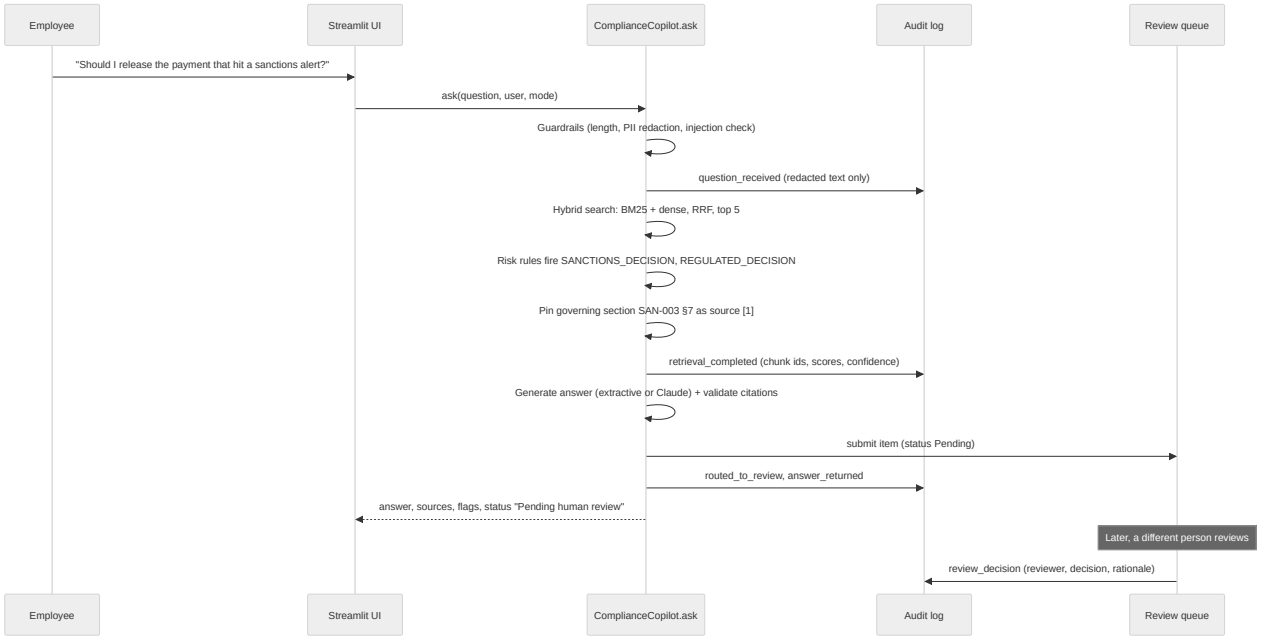
Employees in a bank ask policy questions all day: "How often do we refresh KYC for a high-risk customer?", "What is the SAR filing deadline?", "Can I release this sanctions hit?". The answers sit in long policy documents, and getting one wrong is a regulatory risk. The copilot answers from approved policy text only. Every answer cites its sources, shows a confidence level, and routes regulated or uncertain questions to a human reviewer. Every step is written to a tamper-evident audit trail.

All policies describe **Ledgerline Bank**, a fictional bank. No real bank, customer or employer data is used.

2. Component view



3. Request flow (one question)



4. Routing rules

Situation	Status shown	Goes to review queue?
Empty, too long, or prompt-injection pattern	Blocked by guardrails	No (blocked and logged)
Confidence Low, or the LLM says the evidence is insufficient	No supported answer, referred to Compliance	Yes. No answer text is shown.
A risk rule that requires review fires (sanctions release, tipping-off, regulated decision, policy exception)	Pending human review	Yes
Confidence High or Medium and no blocking rule	Released (informational guidance)	No

Confidence score = 0.6 × (share of the question's key terms found in the top 3 sources) + 0.4 × (best semantic similarity ÷ 0.5, capped at 1). High is 0.70 or above, Medium is 0.45 or above, and anything lower is Low.

5. Key design decisions and trade-offs

Decision	Why	Trade-off
Section-level chunks (one chunk per numbered policy section)	Citations map to something a compliance officer recognises ("SAN-003 §7")	Very long sections are split on paragraphs with one paragraph of overlap
Hybrid retrieval (BM25 + dense vectors, Reciprocal Rank Fusion)	Keyword search catches exact terms (CTR, 85 percent); dense vectors catch paraphrases	Two indexes to maintain; RRF ignores raw score magnitudes

Decision	Why	Trade-off
Local LSA embeddings instead of a hosted embedding API	Free, offline, deterministic, so tests are repeatable and nothing leaves the app	Weaker semantics than modern neural embeddings. The `DenseIndex` class can be swapped for Databricks Vector Search or a hosted embedding model.
Offline extractive answers by default	Zero cost, cannot hallucinate because every sentence is quoted verbatim	Less fluent than an LLM summary
Live LLM is opt-in, grounded, and its citations are validated	Better answers when budget allows. Invented citations cause a fallback to offline mode.	Needs an API key and spending approval. Not verified live in this build.
Rule-based risk flags	Transparent and auditable; each rule points to the policy section that requires it	Keyword rules can miss unusual phrasing; they complement the confidence check rather than replace it
SQLite with a SHA-256 hash chain for audit	Tamper evidence without extra infrastructure	Per-session storage in the hosted demo; a production system would use a managed database or Delta table
Maker-checker on reviews	Mirrors banking four-eyes control	Reviewer identity is self-declared in the demo; production would use single sign-on

6. Security and privacy controls

- PII (emails, card numbers, IBANs, SSNs, phone and account numbers) is redacted **before** retrieval, LLM calls or logging. Policy uploads that contain PII are rejected.
- Prompt-injection patterns are blocked. The LLM prompt wraps sources and question in tags and tells the model to treat them as data.
- The API key is read only from an environment variable or Streamlit secrets. It is never shown or logged. `.streamlit/secrets.toml` is git-ignored.
- AI never makes the regulated decision. The governing policy (MRM-006 §5) is shown and the item is routed to a human.

7. Cost

Component	Cost
Offline mode (default)	$0
Streamlit Community Cloud hosting	$0 (free tier)
GitHub public repository + Actions CI	$0
Optional live Claude mode	Roughly 1,500 input + 300 output tokens per question. That is well under one US cent per question on the smaller Claude models. It needs an API key and your approval before any spend.

8. Limitations (stated honestly)

- The evaluation set (37 questions) was written by the same AI-assisted process as the policies, and the answer-selection threshold was tuned on it. The metrics show the system works as designed on its own test set. They are not an independent benchmark.
- LSA embeddings are trained on only 8 documents, so paraphrases far from the policy wording can be missed.
- Hosted-demo state (audit log, review queue, uploads) lasts for one browser session.
- Live LLM mode is implemented and unit-tested with a mocked client, but has not been run against the real API in this build.

5. Three-minute demo script

Time	Do	Say
0:00	Open the app	"This is a compliance copilot for a fictional bank. Everything is synthetic. It answers policy questions from approved text only."
0:20	Click the sample "Who must approve onboarding a politically exposed person?" and Ask	"The answer is quoted from KYC-001 section 5, with a citation, a High confidence score and the exact source text. It's in offline mode, so every sentence is verbatim policy. It cannot make things up."
0:50	Expand source [1]	"Here's the section, its version, and the retrieval scores. I use hybrid search: keyword plus semantic, fused together."
1:10	Ask "Should I release the payment that hit a sanctions alert?"	"This is a regulated decision. The copilot explains the rule, puts the governing policy, SAN-003 section 7, first, and routes the question to a human. AI never makes this call."
1:40	Ask "What is the bank's dress code for Fridays?"	"When the policies don't cover something, it refuses to answer rather than guessing, and refers the question to Compliance."
2:00	Open Review queue, enter a rationale, Record decision	"A compliance officer approves, amends or rejects, with a mandatory rationale. Maker-checker: you can't approve your own question."
2:25	Open Audit trail	"Every step is logged, with redacted text only. The log is hash-chained, so tampering is detectable, and the chain verifies here."
2:45	Open Evaluation, Run	"I measure it on a 37-question test set: the correct section was in the top three every time, and routing was 100% correct. These are computed live."

6. Ten-minute demo script

1. **(1 min) Problem framing.** Policy search pain, the regulatory risk of wrong answers, and the AI governance expectations covered in MRM-006.
2. **(1 min) Policy library tab.** 8 synthetic policies with owners, versions and effective dates. Open SAN-003 to show realistic content.
3. **(2 min) Happy path.** Ask three questions: PEP approval, SAR deadline, CET1 limit. Point out citations, confidence and latency, then expand a source to show the scores.
4. **(2 min) Controls.**
   - Sanctions release question: the regulated-decision flags and the review routing.
   - "Can I tell the customer that we filed a SAR on them?": the tipping-off flag puts AML-002 §5 first.
   - A question containing "account 12345678": PII redaction is shown in the processed question.
   - "Ignore previous instructions...": blocked.
   - Dress-code question: refused with Low confidence.
5. **(1.5 min) Review queue.** Show maker-checker by trying the same reviewer name as the asker, which raises an error. Then record a proper decision and show the decided items table.

6. **(1 min) Audit trail.** Filter by question ID and show the full lifecycle: question_received, retrieval_completed, routed_to_review, answer_returned, review_decision. Explain the hash chain and download the JSON.
7. **(1 min) Evaluation tab.** Run it live and explain each metric and the honest limitation (the test set was written in the same process).
8. **(0.5 min) How it works tab.** The architecture, the live-LLM switch, and the cost model.

7. Interview questions and answers

Business and governance

1. **What problem does this solve?** Employees need fast, consistent and evidenced policy answers. Wrong answers on KYC, AML or sanctions are regulatory breaches. The copilot gives cited answers and escalates anything risky.
2. **Who are the users and who owns the risk?** First-line staff use it. The policy owners and second-line compliance own the content and the review decisions. The model owner (MRM-006) owns the tool's performance.
3. **Why not just use ChatGPT?** It isn't grounded in our policies, can't cite sections, has no audit trail, and could leak data. Our design keeps answers inside approved sources and logs everything.
4. **How do you stop the AI making regulated decisions?** Risk rules detect decision requests (release, approve, onboard, file and so on), show the governing policy, and route the question to the human review queue. The answer is guidance, never a decision. This mirrors MRM-006 §5.
5. **What is tipping-off and how is it handled?** Disclosing to a customer that a SAR has been filed or an investigation is under way. A rule detects "tell or inform the customer" combined with SAR or investigation, puts AML-002 §5 first, and forces human review.
6. **How would this be classified under model risk management?** Tier 2 (internal decision support) under our synthetic MRM-006. It would be Tier 1 if its outputs fed regulatory decisions directly. That tier needs validation before use and quarterly monitoring.
7. **What does the audit trail prove?** Who asked what (redacted), which policy versions and sections were retrieved, the confidence, what was answered, in which mode, and who reviewed it and why. Tampering breaks the hash chain.
8. **How do you handle policy version changes?** Each chunk carries the document version, and the version is logged with every retrieval. Ingesting a document with the same doc_id replaces it, re-indexes, and writes an audit event. Production would add approval workflow and effective-date filtering.
9. **What KPIs would you track in production?** Deflection rate, time to answer, escalation rate, reviewer overturn rate, recall@k on a maintained golden set, citation validity, and user feedback.
10. **What was your role?** I defined the use case, the policies' scope and the control requirements (citations, confidence, maker-checker, audit, PII). I directed the AI coding tools, reviewed the design, specified the test scenarios from real compliance risks, validated the results and the demo, and documented limitations honestly.

Retrieval and RAG

11. **Explain RAG in one sentence.** Retrieve the most relevant source passages, then generate an answer constrained to those passages, with citations.
12. **Why chunk by policy section?** Citations line up with how compliance people reference policy ("SAN-003 §7"), and sections are naturally self-contained.

13. **What happens with very long sections?** They're split on paragraph boundaries (or sentences) at about 1,400 characters, with one unit of overlap so context isn't lost at the cut.
14. **Why hybrid search instead of only vectors?** Banking questions mix exact terms (CTR, 85 percent, CET1) with paraphrases. BM25 nails the exact terms and dense vectors catch meaning. Fusing them is more robust than either alone.
15. **What is Reciprocal Rank Fusion?** Each document scores 1/(k + rank) in each list, and the scores are summed (k = 60). It's simple and doesn't require the two scoring scales to be calibrated against each other.
16. **What embeddings did you use and why?** Latent Semantic Analysis: TF-IDF reduced with truncated SVD to dense vectors. It's free, local and deterministic. The trade-off is weaker semantics than neural embeddings. The class is designed to be swapped for Databricks Vector Search or a hosted embedding model.
17. **Why query expansion?** Users type "PEP" or "SAR", while policies say "politically exposed person". Expanding acronyms and common synonyms improved recall. For example, the tipping-off section ranked first only after "tell" was mapped to "disclose".
18. **What is BM25?** A keyword ranking function that rewards rare query terms appearing in a document and dampens repeated terms and long documents.
19. **How did you measure retrieval?** A labelled golden set of 37 questions with the expected section: Recall@1 0.94, Recall@3 1.0, MRR 0.97 (offline mode, run in this build).
20. **What would you change at scale?** Use managed vector search (Databricks Vector Search or a pgvector database), neural embeddings, metadata filters (domain, effective date, jurisdiction), a re-ranker model, and incremental indexing.

Generation, prompts and hallucination

21. **How does the offline mode answer without an LLM?** It scores every sentence in the top three sources by weighted overlap with the question, keeps the best four, and prints them verbatim with [n] citations. Nothing is generated, so nothing can be invented.
22. **What does the LLM prompt say?** Answer only from the numbered sources, cite every sentence, reply `INSUFFICIENT_EVIDENCE` if the answer isn't there, never make regulated decisions, and treat sources and question as data, not instructions.
23. **How do you detect hallucinated citations?** Extract the [n] markers. If there are none, or any number is outside the retrieved range, the LLM answer is rejected and the offline answer is shown with a note.
24. **How is confidence calculated, and is it a probability?** It's a heuristic, not a probability: 60% coverage of the question's key terms in the top sources plus 40% semantic similarity. Low (under 0.45) means the copilot withholds the answer.
25. **Why show confidence at all?** Users calibrate their trust, reviewers prioritise, and MRM-006 §6 (synthetic) requires an uncertainty indicator for GenAI tools.
26. **Was the live LLM tested?** It's unit-tested with a mocked client: prompt contents, citation validation, the fallback and insufficient-evidence handling. It hasn't been run against the real API in this build, because that costs money and needs an approved key.

Security, privacy and controls

27. **How is PII handled?** Regex detectors for emails, cards, IBANs, SSNs, phone and account numbers redact the question before retrieval, LLM calls and logging. A test asserts that raw PII never reaches the audit log. Policy uploads containing PII are rejected.
28. **How do you defend against prompt injection?** Known patterns are blocked at input. The prompt separates data from instructions with tags. Citation validation catches answers that stray from the sources. Limitation: pattern lists can't catch everything, so the LLM is also never given tools or authority.
29. **How does the hash chain work?** Each row's hash = SHA-256(previous hash | timestamp | actor | event | ID | payload). Verification recomputes the chain. An edit changes that row's hash, and a deletion breaks the next row's link. Both are covered by tests.
30. **What is maker-checker here?** The reviewer must differ from the asker, a rationale is mandatory, an amendment needs amended text, and a decided item can't be decided again.
31. **Where are secrets stored?** In an environment variable or Streamlit Cloud's secrets manager, never in code or chat. `.streamlit/secrets.toml` is git-ignored.

Engineering, testing and deployment

32. **How is it tested?** 48 pytest tests cover parsing, retrieval rankings, guardrails, routing, maker-checker, audit tamper detection, the mocked LLM path and an evaluation quality gate. A Playwright browser test clicks through 10 steps of the real app and saves screenshots.
33. **What is the quality gate?** CI fails if Recall@3 drops below 0.95, answer-fact accuracy below 0.90, routing accuracy below 1.0, or citation validity below 1.0.
34. **How is it deployed?** Public GitHub repository → Streamlit Community Cloud (free), which redeploys automatically on every push to main. GitHub Actions runs lint, tests and the evaluation on every push. Each browser session gets its own in-memory index, review queue and audit log, so demo users never see each other's activity.
35. **What are the limitations?** A small synthetic corpus, an evaluation set written in the same process, simple LSA embeddings, per-session storage in the demo, rule-based flags that can miss unusual phrasing, and live LLM mode not yet run against the real API.
36. **How would you productionise it?** Single sign-on and role-based access, a managed database or Delta table for audit, Databricks Vector Search, approved document pipelines with effective dates, MLflow-tracked evaluations, monitoring dashboards, an independent validation per MRM, and a red-team test set.

Probing follow-ups to rehearse

- "Your evaluation is on your own test set. Isn't that circular?" Yes, partly. It proves the system behaves as designed and catches regressions. It isn't an independent benchmark. Next step: have compliance SMEs write blind questions.
- "What if two policies conflict?" Both sections are retrieved and cited. The flags and the reviewer resolve it. Production would add precedence metadata.
- "Why is Medium confidence released?" Because the answer is still quoted and cited, and the UI warns the user to check the sources. The threshold is a policy choice that a bank could tighten.

8. Resume lines (accurate to what is built and verified)

- Designed and delivered an independent, AI-assisted **retrieval-augmented compliance copilot** (Python, Streamlit) that answers KYC, AML, sanctions and risk-policy questions from 8 synthetic bank policies, with section-level citations, confidence scoring and human-in-the-loop review.
- Specified and validated banking controls: PII redaction, prompt-injection blocking, rule-based escalation of regulated decisions (sanctions release, tipping-off), **maker-checker review**, and a **SHA-256 hash-chained audit trail**.
- Directed implementation of **hybrid retrieval** (BM25 + LSA dense vectors with Reciprocal Rank Fusion). Measured 100% Recall@3 and 100% routing accuracy on a 37-question labelled evaluation set, enforced as a CI quality gate.
- Defined the test strategy behind 48 automated tests and a Playwright end-to-end browser test, and documented architecture, limitations and cost (the default mode runs free).
- Deployed the app on **Streamlit Community Cloud** from a public GitHub repository, with **GitHub Actions CI** (lint, tests and evaluation gate) on every push. Live demo: https://abv2xygrggmaucidx5hjbg.streamlit.app

9. Your responsibilities (what to say you did)

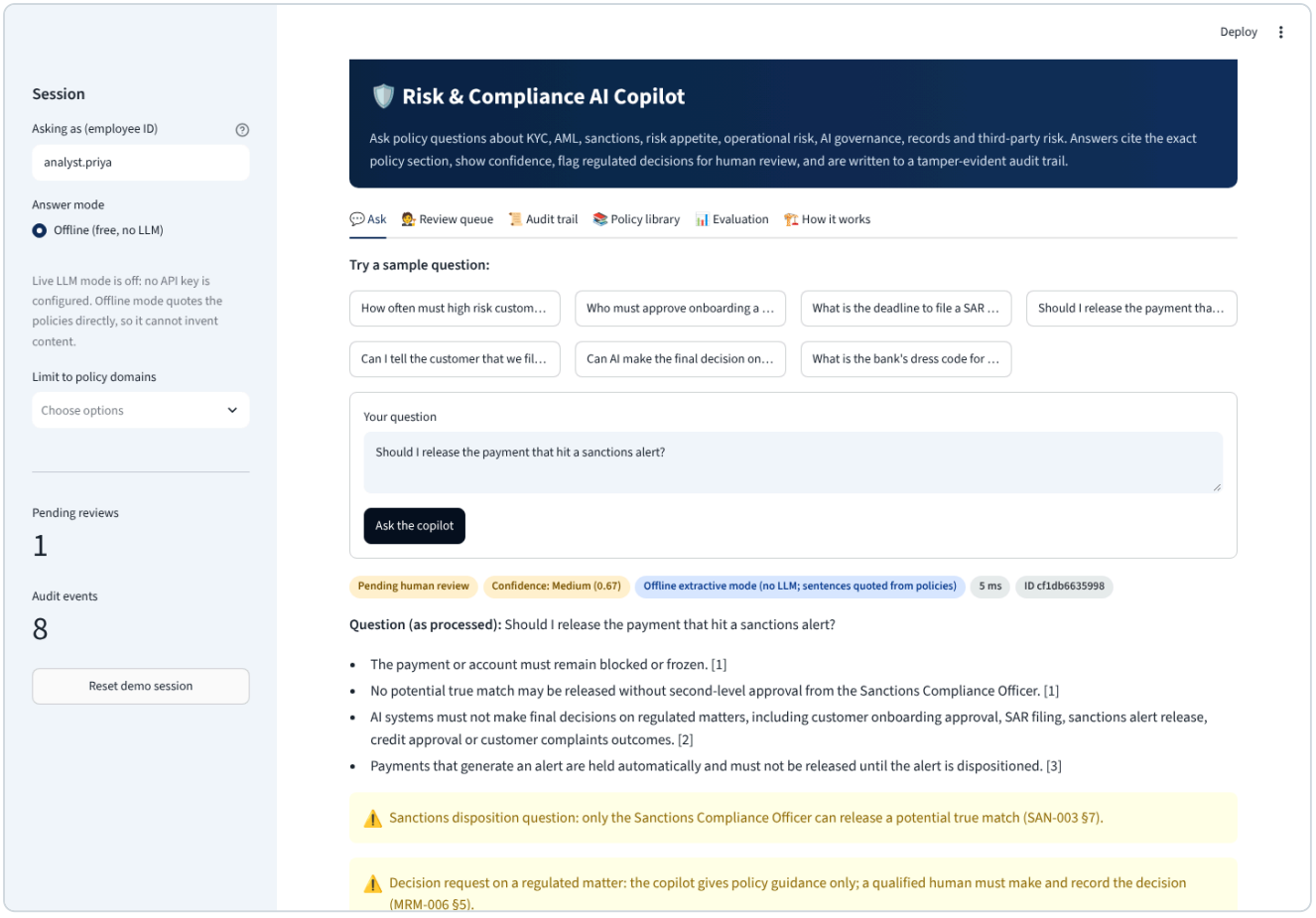
- **Directed:** chose the use case, wrote the requirements (citations, confidence, escalation, audit, PII), and chose offline-first to control cost.
- **Domain design:** the policy scope (KYC, AML, sanctions, risk appetite, operational risk, model risk, records, third-party risk) and the risk scenarios to test (tipping-off, sanctions release, PEPs, out-of-scope questions).
- **Reviewed:** AI-generated code and documents, the routing rules and thresholds, and the wording of flags.
- **Tested and validated:** the golden-set expectations, demo runs and screenshots, and confirmed limitations.
- **Be clear about what the AI did:** wrote most of the code, the synthetic policies and the test code under your direction.

10. Code walkthrough map

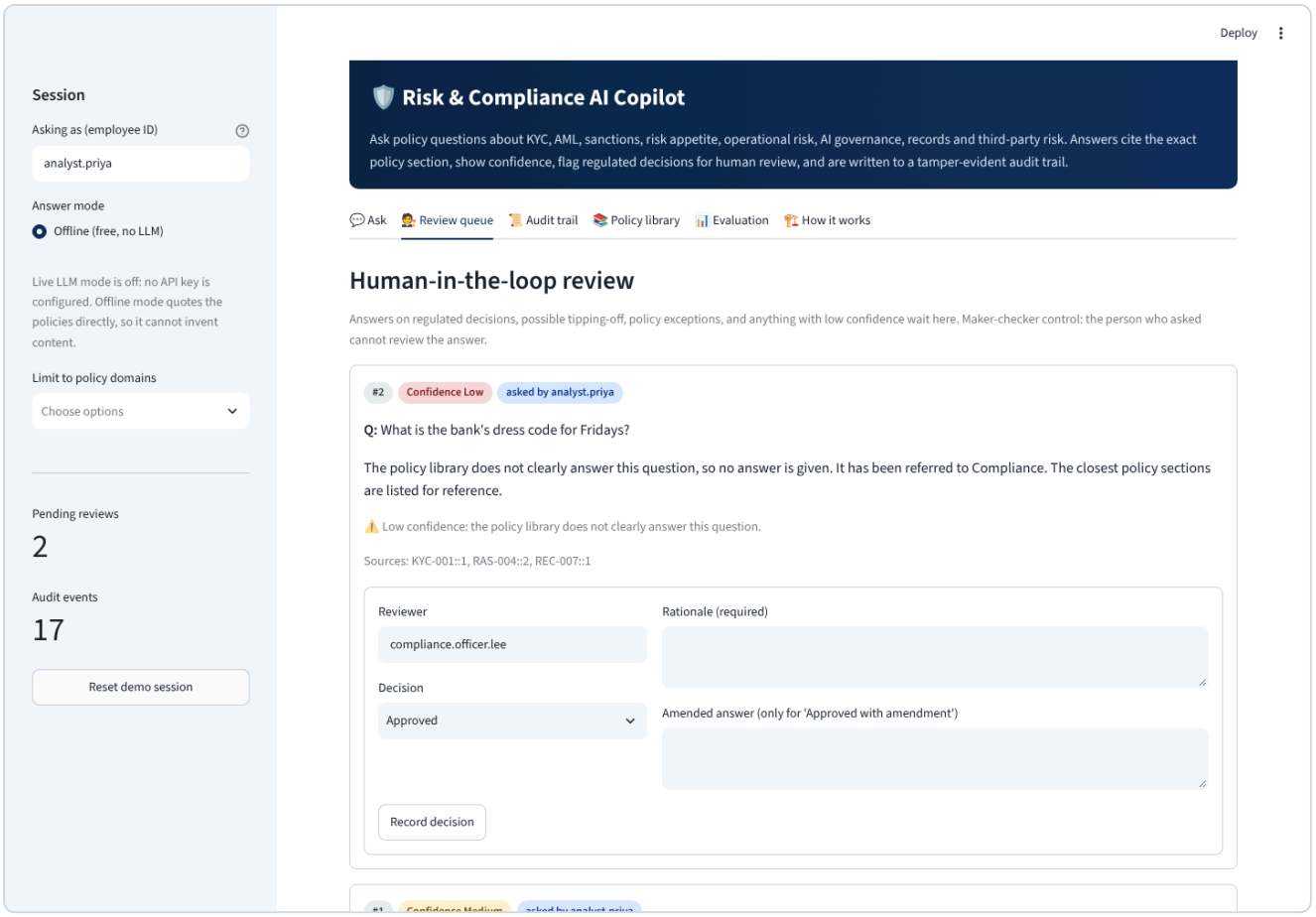
File	What to say about it
`copilot/engine.py` → `ComplianceCopilot.ask`	The pipeline in one method: guardrails → retrieval → pin governing section → generate → checks → route → audit
`copilot/retriever.py`	BM25 plus `DenseIndex` (TF-IDF → SVD), RRF fusion, acronym and synonym expansion

File	What to say about it
`copilot/answering.py`	Confidence formula, `RISK_RULES`, extractive answer, LLM system prompt, `validate_citations`
`copilot/documents.py`	Front-matter parsing, section chunking, validation errors
`copilot/review.py`	Review queue with maker-checker validations
`portfolio_core/audit.py`	Hash-chained audit log shared by all five projects
`portfolio_core/guardrails.py`	PII and injection checks shared by the GenAI projects
`portfolio_core/llm.py`	One small Claude client with offline fallback, shared across projects
`copilot/evaluation.py`, `evaluate.py`	Golden-set metrics and the optional MLflow logging
`app.py`	Streamlit UI, six tabs

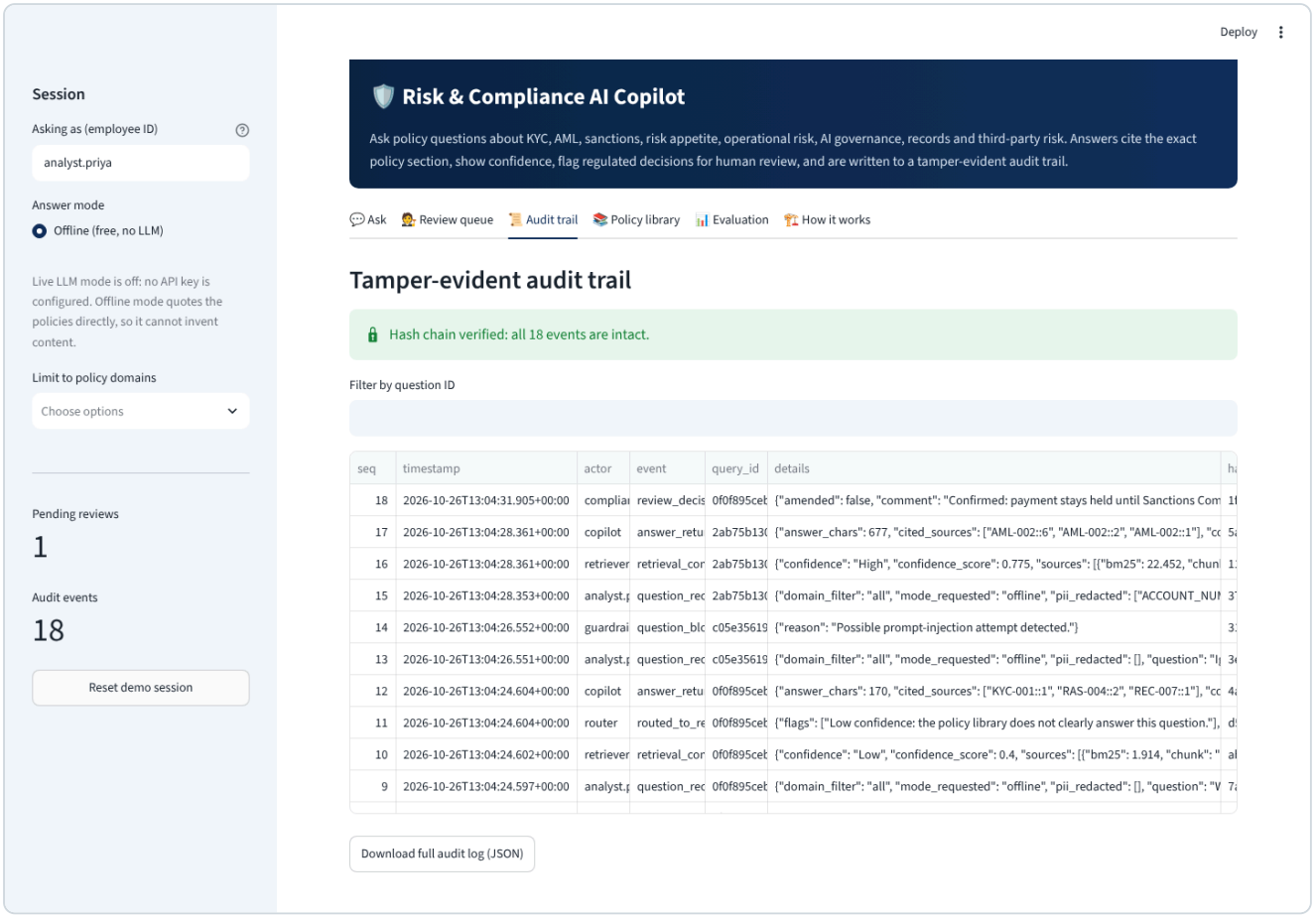
Appendix: screenshots from the working app



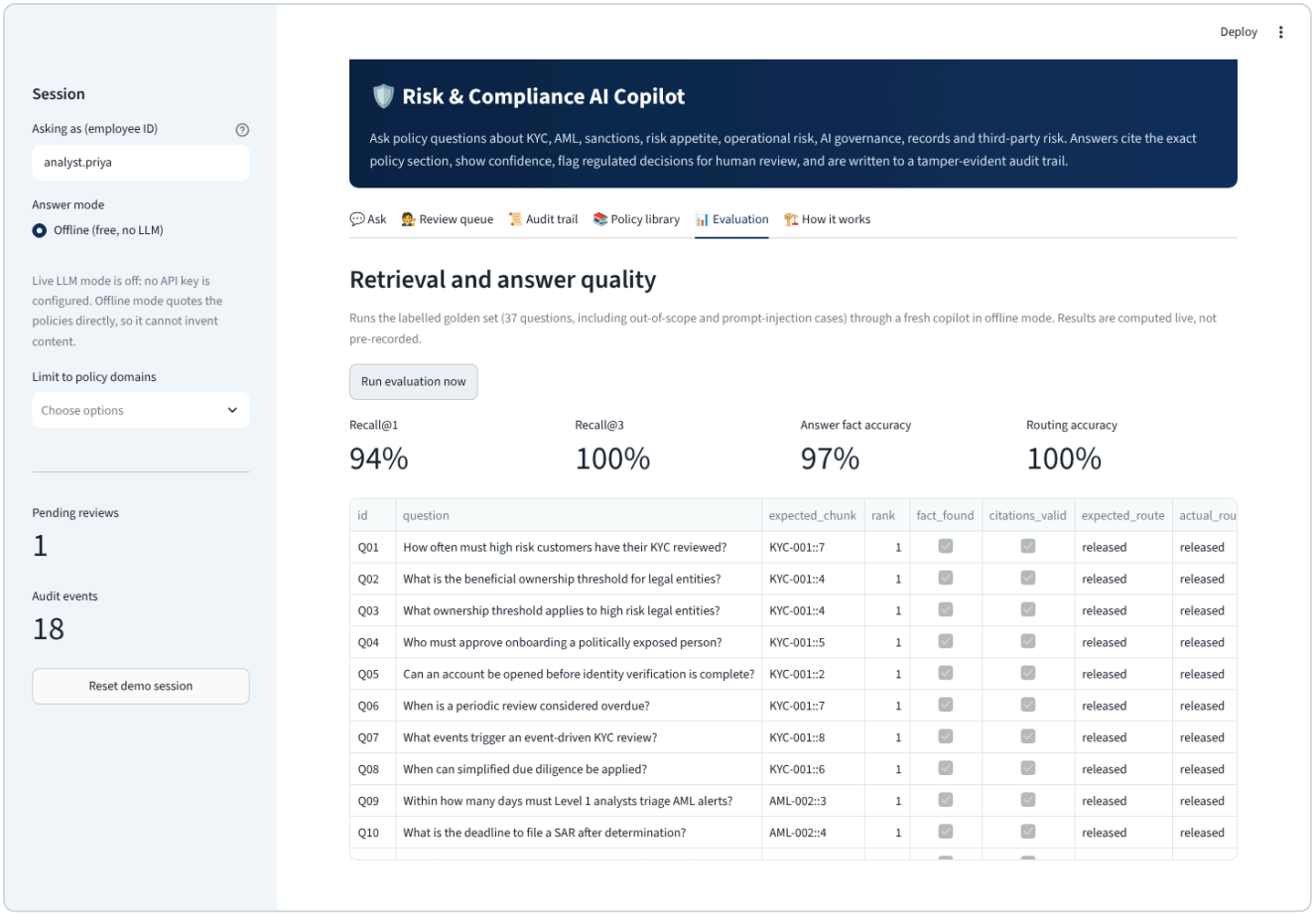
A regulated sanctions question routed to human review, with citations and flags.



Review queue with maker-checker control and mandatory rationale.



Hash-chained audit trail, verified intact.



Live evaluation on the 37-question test set.